Appello del 22 Settembre 2014



Politecnico di Milano

Anno accademico 2010-2011

Ingegneria del Software

Cognome:		
Nome:		Matricola:
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 5:

Esercizio 1

Si deve progettare una classe `Clock`, che realizza oggetti che rappresentano ora, minuto e secondo. La classe offre le operazioni `click(x)`, che fa avanzare il tempo di `x` secondi, e le operazioni getter `getHour`, `getMin` e `getSec`, che restituiscono l'ora, il minuto e il secondo di un oggetto `Clock`. La classe implementa pure l'interfaccia `Comparable`, di cui si riporta la definizione:

```
public interface Comparable<T> {
  //@ensures (* if this<arg then \result ==-1
  //@  else if this == arg then \results == 0,
  //@     else \result == 1 *);
  public int compareTo(T arg);
}
```

Domanda 1

Fornire la specifica JML del metodo `click`. Il metodo non solleva eccezioni.

Domanda 2

Definire una rappresentazione (rep) per la classe. Specificare l'eventuale rep invariant e fornire un'implementazione della funzione di astrazione, come implementazione dell'operazione `toString`.

Domanda 3

Definire un'implementazione del metodo `compareTo`.

Esercizio 2

Si consideri la seguente specifica di una classe Lista. Una Lista è una sequenza di elementi che consente ricerca, inserimento ed eliminazione. (Per l'interfaccia Comparable, si faccia riferimento all'Esercizio 1)

```
public class Lista<T extends Comparable<T>> {
/*OVERVIEW: Collezione mutabile di oggetti, di tipo T,
 * organizzati in una sequenza.
 * Oggetto tipico di dimensione n: x0 x2 ... xn-1,
 * in cui x0 e' il primo elemento e xn-1 l'n-esimo.
 * Rimozione e inserimento di elementi possono avvenire
 * in una posizione qualunque.
 */

//@ensures (*inizializza this alla  Lista vuota*);
public Lista()

//osservatori puri:

//@ensures (* \result == numero di elementi di this *)
public /*@ pure @*/ int size()

//@ensures (* \result <==> c'e' elemento equals(x) in this *)
public /*@ pure @*/ boolean  contains(T x)

//@requires i>=0 && i < size();
//@ensures (*  \result == l'elemento di this di posizione i-esima;
public /*@ pure @*/ T get(int i )

//mutators:

//@requires size()>0;
//@ ensures (*  elimina e restituisce il massimo elemento
//@ (secondo compareTo) in this *);
public T extractMax()


//@ensures (* aggiunge x in una qualunque posizione della lista *);
public void insert(T x)


//@requires size()>0;
//@ ensures (* elimina da this  una comparsa di un elemento equals(x) *)
public void remove(T x)
}
```

Domanda 1

Specificare in JML i metodi `insert` e `extractMax`, evitando di inserire vincoli non richiesti. Ad esempio, non si forzi la scelta di una posizione specifica in cui inserire l'elemento nel caso dell'operazione `insert`.

Domanda 2

Si consideri una classe `ListaSenzaRipetizioni`, definita come `Lista`, ma con il vincolo che non possono esistere elementi ripetuti. Pertanto l'operazione `insert` non inserisce un elemento se questo è già presente nella lista.

1. È possibile definire `ListaSenzaRipetizioni` come sottoclasse di `Lista` o viceversa rispettando il principio di sostituzione?
2. Fornire la specifica JML dell'operazione insert.
3. Definire un invariante astratto per la classe che specifica l'assenza di elementi ripetuti.

Esercizio 3

Si supponga di disporre di una classe con metodi che accettano parametri e restituiscono risultati in unità di misura cosiddette *British Imperial*, cioè piedi, miglia, etc. Si vuole riusare questa classe, ma utilizzando parametri e risultati che siano espressi nel sistema *Metrico*. Spiegare se e come il problema può essere risolto utilizzando design pattern opportuni. Fornire anche una bozza della soluzione usando un diagramma delle classi UML.

Esercizio 4

Si consideri il seguente frammento di programma che opera su interi:

```
1. ''legge n da input'';
2. if (n>0) {
3.            j = n*(n-1);
4.            while (j>0) {
                    ----//qui codice che non cambia n e j
5.                  j--;
6.                  if (j==0) exit;
7.            }
8.  ----;
9. };
```

1. Si definisca un insieme minimo di casi di test che coprano tutte le istruzioni (si considerino le parti tralasciate `----` come se fossero una singola macro-istruzione);
2. Si definisca un insieme minimo di casi di test che coprano tutti i branch.
3. Si consideri un cammino che esegue il ciclo due volte. Si scriva l'espressione logica che deve essere soddisfatta per la percorrenza di detto cammino e si identifichi un valore di input che causa la percorrenza dello stesso.

Esercizio 5

Si consideri una tabella di dati, realizzata come singleton. Ogni riga della tabella contiene una chiave intera ed una stringa come valore. La tabella offre le seguenti operazioni:

1. **searchAndGet** riceve come parametro una chiave e restituisce tutte le stringhe memorizzate nella tabella di dati con quella chiave, cancellandola dalla tabella stessa.

2. **insert** riceve un dato (stringa), con chiave associata, e lo memorizza.

La tabella viene usata in un ambiente concorrente da più thread. Poiché essa ha una dimensione predefinita, quando risulta piena, l'invocazione dell'operazione `insert` sospende il task che invoca l'operazione in attesa che venga liberato spazio dall'invocazione di `searchAndGet`.

Si fornisca un'implementazione Java del singleton `Tabella`.